1 Message Integrity and Hash Functions

1.1 Integrity versus secrecy; message and digest ▸

▸ **Likely this year:** "Why is a hash, not encryption, used for integrity?" and "how is integrity checked with a digest?" (2–3 marks).

Every scheme so far gave *secrecy*: Eve cannot read the message. Many documents need the opposite guarantee. Alice's will, for example, need not be encrypted, because after her death anyone may read it; what must be preserved is its *integrity*, meaning that nobody can change its contents without the change being noticed. On paper this is achieved by putting Alice's fingerprint at the bottom of the document: Eve can read the will, but she cannot alter it or write a false one, because she cannot forge the fingerprint.

The electronic equivalent of the document–fingerprint pair is the **message–digest pair**. The message is passed through a **cryptographic hash function** h, which maps a message of any length to a short fixed-length string $h(M)$, called the **message digest**. The digest is a compressed image of the message that plays the role of the fingerprint. To check integrity later, the receiver runs the same hash function on the message he now holds and compares the new digest with the stored one: if the two digests are equal, the message has not been changed; if they differ, the message was changed and is discarded.

Why a hash and not encryption. Encryption hides the message but does not stop Eve from altering the ciphertext; flipping a bit of a one-time-pad or stream-cipher ciphertext flips the same bit of the plaintext, and Bob has no way to tell a changed plaintext from a genuine one. A hash works the other way round: it hides nothing (the message can be public), but since any change to the message produces a different digest, any change is detected. The two services are therefore independent, and a message can need one, the other, or both.

The digest itself must be safe. The check only works if the stored or received digest is genuine. If Eve can replace both M and $h(M)$, she simply sends M' with $h(M')$ and the comparison passes. Hence the digest is kept in a safe place or sent over an authenticated channel; binding the digest to a secret key gives the MAC of the next lecture.

1.2 The three security criteria ▸

▸ **Likely this year:** "Define and differentiate preimage, second preimage and collision resistance" (3–4 marks), possibly with a figure of what Eve is given and must find.

A cryptographic hash function must make three kinds of forgery infeasible. They differ in how much Eve is given at the start, and therefore in how much freedom she has.

Preimage resistance. Given a hash function h and a digest $y = h(M)$, it must be extremely difficult for Eve to find *any* message M' with $h(M') = y$. If h were not preimage resistant, Eve could intercept just the digest $h(M)$, construct a message M' that hashes to it, and send M' to Bob as if it were M. In other words, the hash must be one-way.

Second preimage resistance. Given a specific message M *and* its digest $h(M)$, it must be extremely difficult to find another message $M' \neq M$ with $h(M') = h(M)$. This is the criterion that stops forgery of a particular message: if Alice sends M and $h(M)$ to Bob, Eve cannot create a different message with the same digest and substitute it. Eve here has more information than in the preimage case (she also knows one valid preimage), but she still has to hit one fixed target value.

Collision resistance. It must be extremely difficult to find *any* two messages $M \neq M'$ with $h(M) = h(M')$. Here Eve is given nothing and is free to choose both messages from scratch; such a pair is called a **collision**. This is the danger when Eve prepares two documents herself (say a fair contract and an unfair one with the same digest), gets Alice to approve the fair one, and later swaps in the other.

Criterion	Eve is given	Eve must find	Attack it stops
Preimage resistance	h and a digest $y = h(M)$	any M' with $h(M') = y$	preimage
Second preimage resistance	h, a message M and $h(M)$	$M' \neq M$ with $h(M') = h(M)$	second preimage
Collision resistance	only h	any pair $M \neq M'$ with $h(M) = h(M')$	collision

Collision resistance implies second preimage resistance. Suppose Eve had a method that, given any M, produces a second preimage M'. She could then pick any M she likes, run the method, and output the pair (M, M'), which is a collision. So a function that is *not* second-preimage resistant cannot be collision resistant; equivalently, a collision-resistant function is automatically second-preimage resistant. The converse fails, because finding a collision is the easier task (Eve chooses both sides), and the birthday analysis below shows by how much: a collision costs about $2^{n/2}$ hash evaluations, a second preimage about 2^n.

Why a compressor or a checksum is not a cryptographic hash. A lossless compression method such as ZIP or StuffIt also shrinks the message, but it is reversible: anyone can decompress the output and get the original, so it is not even a function to a fixed-length output, and it fails preimage resistance trivially. A checksum (adding up the bytes, a CRC) is fixed-length but linear and easy to invert: given a target checksum, Eve can compute what to

change in any message to reach it, so it is neither preimage nor second-preimage resistant. Checksums detect accidental errors, not deliberate ones.

★ MUST KNOW : the three criteria

Preimage: given y, find any M' with $h(M') = y$. **Second preimage**: given M, find $M' \neq M$ with $h(M') = h(M)$. **Collision**: find any $M \neq M'$ with $h(M) = h(M')$.
Cost for an ideal n-bit hash: preimage and second preimage $\approx 2^n$, collision $\approx 2^{n/2}$. Collision resistance $\Rightarrow$ second preimage resistance.

1.3 The random oracle model ▸

▸ **Likely this year:** "Explain the random oracle model; why can the oracle not use a formula?" (2–3 marks).

To analyse attacks we need a model of a perfect hash function, against which real designs are compared. The **random oracle model** (Bellare and Rogaway, 1993) describes such an ideal function as an oracle that keeps a table of the messages it has already hashed and obeys three rules:

1. When a *new* message of any length is given, the oracle creates a fixed-length digest that is a truly random string of 0s and 1s, gives it out, and records the message and digest in its table.
2. When a message is given for which a digest already exists, the oracle simply returns the digest in the table. This makes the function deterministic: the same message always gets the same digest.
3. The digest for a new message must be chosen independently of all previous digests. This implies that the oracle cannot use a formula or an algorithm to calculate the digest.

Example (slides 20–23). An oracle with 16-bit digests already holds three entries. It is asked to hash AB1234CD8765BDAD, which is not in its table, so it flips a fair coin 16 times and gets HHTHHHTTHTHHTTTH. Reading H as 1 and T as 0 gives 1101 1100 1011 0001 = DCB1, which it returns and records. When it is then asked for 4523AB1352CDEF45126, which is already in the table, it does not flip again but returns the stored digest 13AB.

Message	Digest
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB1234CD8765BDAD	DCB1 (new)
AB45CD1048765412AAAB6662BE	A38B

Why a formula violates rule 3. Suppose the oracle used $h(M) = M \bmod n$ and had already issued $h(M_1)$ and $h(M_2)$. For the new message $M_3 = M_1 + M_2$ anyone can predict the digest without asking, since

$$h(M_3) = (M_1 + M_2) \bmod n = [\, h(M_1) + h(M_2) \,] \bmod n.$$

The digest of M_3 is therefore not independent of the earlier digests, which breaks rule 3. Real hash functions are algorithms, so they can only approximate the oracle; the model tells us the best security an n-bit design can hope for.

What the model gives an attacker. With an n-bit digest there are $N = 2^n$ possible digests, and each new message receives one of them uniformly at random. Some values may be chosen several times and some never. Eve knows the algorithm and n, so every attack reduces to drawing random digests and waiting for a match, which is exactly a birthday problem.

1.4 The four birthday problems ▸

▸ **Likely this year:** Derive the probability formula for one problem and get k for $P = 1/2$; or "how many people for a shared birthday" (23).

All four problems use a random variable that is uniform over N values (0 to $N - 1$; $N = 365$ for birthdays) and ask for the minimum number of samples k that makes an event *likely*, meaning probability $P \geq 1/2$. Every derivation uses the approximation $1 - x \approx e^{-x}$ for small x.

Problem 1: a sample equals a predefined value. What is the minimum number of students such that it is likely that at least one has a *predefined* birthday (say 1 January)? A single sample misses the target with probability $1 - 1/N$, and the k samples are independent, so all of them miss with probability $(1 - 1/N)^k \approx e^{-k/N}$. Hence

$$P_1 \approx 1 - e^{-k/N}, \qquad k \approx \ln \frac{1}{1-P} \times N \xrightarrow{P=1/2} k \approx \ln 2 \times N = 0.69N.$$

For $N = 365$: $0.693 \times 365 = 252.99$, so $k = 253$.

Problem 2: a sample equals one chosen from the set. What is the minimum number such that it is likely that at least one student has the same birthday as a student *selected by the professor*? The selected student is one of

the k, so only the other $k-1$ can match, and the problem is Problem 1 with $k-1$ samples:

$$P_2 \approx 1 - e^{-(k-1)/N}, \qquad k \approx \ln 2 \times N + 1 = 0.69N + 1; \qquad N = 365 : \ k = 254.$$

Problem 3: any two samples are equal (the birthday paradox). What is the minimum number such that it is likely that *at least two* students share a birthday? It is easier to count the complement, that all k values are different. The first value is free, the second must avoid 1 value, the i-th must avoid $i-1$ values, so

$$\Pr[\text{all different}] = \prod_{i=1}^{k-1}\left(1 - \frac{i}{N}\right) \approx \prod_{i=1}^{k-1} e^{-i/N} = e^{-(1+2+\cdots+(k-1))/N} = e^{-k(k-1)/2N}.$$

Therefore

$$P_3 \approx 1 - e^{-k(k-1)/2N}, \qquad k \approx \left\{2 \ln \frac{1}{1-P}\right\}^{1/2} N^{1/2} \xrightarrow{P=1/2} k \approx \sqrt{2 \ln 2}\,\sqrt{N} = 1.18\sqrt{N},$$

using $k(k-1) \approx k^2$. For $N = 365$: $1.18 \times 19.1 = 22.5$, so $k = 23$ (exactly, 23 people give $P = 0.507$ and 22 give 0.476). The answer is a "paradox" only because intuition compares each person with *one* date, whereas k people form $k(k-1)/2 = 253$ pairs, each a chance to match.

Problem 4: a sample of one set equals a sample of another. Two classes each have k students. What is the minimum k such that it is likely that some student of the first class shares a birthday with some student of the second? Treat the k values of set 1 as (nearly) distinct. Each value of set 2 then misses all of them with probability $1 - k/N$, and the k values of set 2 all miss with probability $(1-k/N)^k \approx e^{-k^2/N}$. Hence

$$P_4 \approx 1 - e^{-k^2/N}, \qquad k \approx \left\{\ln \frac{1}{1-P}\right\}^{1/2} N^{1/2} \xrightarrow{P=1/2} k \approx \sqrt{\ln 2}\,\sqrt{N} = 0.83\sqrt{N}; \qquad N = 365 : \ k = 16.$$

(Exactly, $k = 16$ gives $P = 0.504$.)

Problem	Probability P	General k	k at $P = \frac{1}{2}$	$N = 365$	Attack
1	$1 - e^{-k/N}$	$\ln \frac{1}{1-P} \cdot N$	$0.69N$	253	preimage
2	$1 - e^{-(k-1)/N}$	$\ln \frac{1}{1-P} \cdot N + 1$	$0.69N + 1$	254	second preimage
3	$1 - e^{-k(k-1)/2N}$	$\{2 \ln \frac{1}{1-P}\}^{1/2} N^{1/2}$	$1.18\sqrt{N}$	23	collision
4	$1 - e^{-k^2/N}$	$\{\ln \frac{1}{1-P}\}^{1/2} N^{1/2}$	$0.83\sqrt{N}$	16	alternate collision

The constants are $0.69 = \ln 2$, $1.18 = \sqrt{2 \ln 2}$ and $0.83 = \sqrt{\ln 2}$. The key comparison is that k **grows like N in Problems 1 and 2 but only like $\sqrt{N}$ in Problems 3 and 4.** Problems 1 and 2 model the preimage and second preimage attacks, Problems 3 and 4 the collision attacks, so a collision attack is far cheaper than a preimage attack on the same hash.

Trap: rounding the constant

With 0.69 exactly, $0.69 \times 365 = 251.85$, which would suggest 252; the table value 253 comes from $\ln 2 = 0.6931$ ($0.6931 \times 365 = 252.99$). Use 0.693 when the answer must be an exact head count. Also do not mix up Problems 1 and 3: "someone has *my* birthday" needs 253 people, "*some two* share a birthday" needs only 23.

WORKED EXAMPLE : birthday numbers for a 16-bit digest

Q. A hash function produces 16-bit digests. Using the birthday results, how many random messages must Eve hash for a probability of at least 1/2 of (a) matching a given digest, (b) finding any two messages with the same digest, (c) a match between two lists of k messages each?

A. $N = 2^{16} = 65\,536$ and $\sqrt{N} = 2^8 = 256$.

(a) Problem 1: $k \approx 0.69 \times 65\,536 \approx 45\,220$ messages (order 2^{16}).

(b) Problem 3: $k \approx 1.18 \times 256 \approx$ **302** messages (order 2^8).

(c) Problem 4: $k \approx 0.83 \times 256 \approx$ **213** messages in each list.

Check: (b) is about 150 times cheaper than (a), as $2^{16}/2^8 = 256$ predicts up to the constants.

1.5 Attacks on an n-bit digest ▸

▸ **Likely this year:** "A hash uses a 64-bit digest; how many digests must Eve create to find a collision (or the original message) with $P > 0.5$, and how long does it take?" (3–4 marks).

Assume the hash behaves like a random oracle with n-bit digests, so its outputs are uniform over $N = 2^n$ values, and that Eve knows the algorithm and n. Each attack below is a loop that creates messages, hashes them, and compares digests; its success probability is the matching birthday problem.

Preimage attack. Eve has intercepted a digest $D = h(M)$ and wants any M' with $h(M') = D$. She creates k messages $M[1], \ldots, M[k]$, computes $T = h(M[i])$ for each, and returns $M[i]$ as soon as $T = D$. The target D is a predefined value, so this is Problem 1: $P = 1 - e^{-k/N}$, and success with $P \geq 1/2$ needs $k = 0.69 \times 2^n$ digests, which is of order 2^n.

Second preimage attack. Eve has intercepted both M and $D = h(M)$ and wants $M' \neq M$ with $h(M') = D$. She creates $k - 1$ further messages and compares each digest with D. The target is one of the k samples (the given M), so this is Problem 2: $P = 1 - e^{-(k-1)/N}$ and $k = 0.69 \times 2^n + 1$, again of order 2^n.

Collision attack. Eve wants any $M \neq M'$ with $h(M) = h(M')$. She creates k messages, stores the digests $D[1], \ldots, D[k]$, and compares each new digest with all earlier ones. Any two equal digests suffice, so this is Problem 3: $P = 1 - e^{-k(k-1)/2N}$ and $k = 1.18 \times 2^{n/2}$, of order only $2^{n/2}$. The square root comes from the number of pairs: k digests give about $k^2/2$ pairs, and once $k^2/2$ is comparable to 2^n a coincidence is likely.

Alternate collision attack. A random collision is usually useless, because the two colliding messages are meaningless strings. A real forger needs a collision between a *genuine* message M (which Alice will approve) and a *bogus* message M' (which Eve will later substitute), both meaningful. Eve therefore makes k variants of each without changing their meaning (an extra space after a word, a synonym, "will not" versus "won't", a redundant phrase): $M_1, \ldots, M_k$ and $M'_1, \ldots, M'_k$. She hashes both lists and looks for $h(M_i) = h(M'_j)$. A match must be *across* the two lists, so this is Problem 4: $P = 1 - e^{-k^2/N}$ and $k = 0.83 \times 2^{n/2}$ variants of each, still of order $2^{n/2}$. Variants are cheap to produce: if a letter has $n/2$ independent places where one of two equivalent forms can be used, it has $2^{n/2}$ variants.

Attack	Birthday problem	k for $P = 1/2$	Order
Preimage	1	0.69×2^n	2^n
Second preimage	2	$0.69 \times 2^n + 1$	2^n
Collision	3	$1.18 \times 2^{n/2}$	$2^{n/2}$
Alternate collision	4	$0.83 \times 2^{n/2}$	$2^{n/2}$

WORKED EXAMPLE : a 64-bit digest (slides 27 and 32)

Q. A cryptographic hash function uses a 64-bit digest. (a) How many digests must Eve create to find the original message with probability more than 0.5, and how long does it take at 2^{30} (about one billion) digests per second? (b) How many digests must she create to find two messages with the same digest with probability more than 0.5, and how long does it take at 2^{20} (about one million) digests per second? Is a 64-bit digest secure?

A. $n = 64$, $N = 2^{64}$.

(a) Preimage attack (Problem 1): $k = 0.69 \times 2^{64} \approx 1.27 \times 10^{19}$ digests.

Time $= 0.69 \times 2^{64}/2^{30} = 0.69 \times 2^{34}$ s $= 1.185 \times 10^{10}$ s $\div (3.156 \times 10^7$ s/year$) \approx$ **376 years**. Secure against the preimage attack.

(b) Collision attack (Problem 3): $k = 1.18 \times 2^{64/2} = 1.18 \times 2^{32} \approx 5.07 \times 10^9$ digests.

Time $= 1.18 \times 2^{32}/2^{20} = 1.18 \times 2^{12}$ s $= 4833$ s $\approx$ **1.34 hours**. **Not** secure against the collision attack, even with a computer a thousand times slower.

Check: the collision attack is cheaper by about 2^{32} in digests, exactly the gap between 2^n and $2^{n/2}$.

What digest length is needed. Since the cheapest generic attack costs about $2^{n/2}$ hash evaluations, an n-bit hash offers only $n/2$ bits of security against collisions. To demand at least 2^{80} work, the digest must have $n/2 \geq 80$, i.e. $n \geq 160$ bits, which is why SHA-1 has 160 bits and the SHA-2 family 224–512 bits. **MD5's 128-bit digest** gives only 2^{64} collision security: $1.18 \times 2^{64} \approx 2.2 \times 10^{19}$ digests, about 640 years on one machine at 2^{30} per second, but well within reach of a large cluster. In addition, structural attacks (Wang et al., 2004) now find MD5 collisions in minutes on a PC, so MD5 must not be used where collisions matter.

WORKED EXAMPLE : new exam-style numerical

Q. A hash function produces 40-bit digests and Eve can compute 2^{20} digests per second. (a) How long does a preimage attack with success probability 1/2 take? (b) How long does a collision attack take? (c) If Eve hashes 2^{20} messages, what is the probability that two of them collide?

A. $N = 2^{40}$.

(a) $k = 0.69 \times 2^{40}$; time $= 0.69 \times 2^{20}$ s $= 723\,517$ s $\approx$ **8.4 days**.

(b) $k = 1.18 \times 2^{20} \approx 1.24 \times 10^6$; time $= 1.18$ s, **about one second**.

(c) $P = 1 - e^{-k(k-1)/2N} \approx 1 - e^{-2^{40}/2^{41}} = 1 - e^{-0.5} =$ **0.39**.

Check: (b) is (a) divided by roughly 2^{20}, consistent with the orders 2^{40} and 2^{20}.

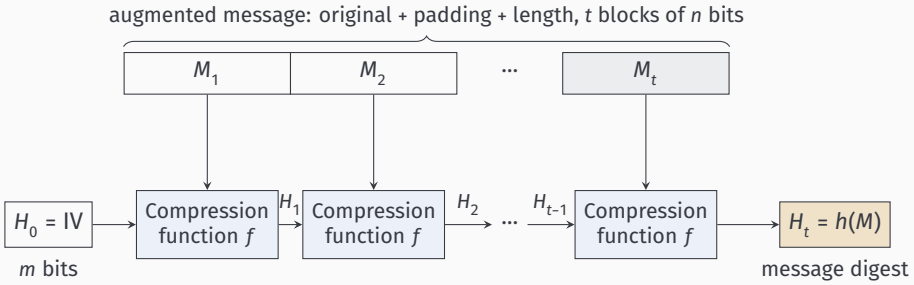
WORKED EXAMPLE : how many variants for an alternate collision

Q. Eve attacks a 64-bit hash with the alternate collision attack. How many variants of the genuine and the bogus letter does she need for success probability about 1/2, and how many two-way choice points must each letter contain? What is her success probability with 2^{32} variants of each?
A. Problem 4: $k = 0.83 \times 2^{64/2} = 0.83 \times 2^{32} \approx 3.56 \times 10^9$ variants of each letter. With m independent places where one of two equivalent forms can be chosen, a letter has 2^m variants, so $m = 32$ choice points per letter are enough ($2^{32} > 0.83 \times 2^{32}$).
With $k = 2^{32}$: $P = 1 - e^{-k^2/N} = 1 - e^{-2^{64}/2^{64}} = 1 - e^{-1} = \mathbf{0.63}$.

1.6 Iterated hash functions: the Merkle–Damgård scheme ▸

▸ **Likely this year:** "Explain the Merkle–Damgård scheme with a diagram" (3–4 marks).

A hash function must accept messages of any length and still produce a fixed-length digest, and designing one function for arbitrary input lengths directly is hard. The standard solution is **iteration**: design a **compression function** f with *fixed-size* input, which compresses an $(n + m)$-bit input (an n-bit message block and an m-bit chaining value) to m bits, and apply it as many times as the message requires. A hash built this way is an **iterated cryptographic hash function**, and the **Merkle–Damgård scheme** is the standard way of doing it.



The scheme works in five steps.

1. The message length and padding are appended to the message to create an **augmented message** whose length is a multiple of n, the block size of the compression function.
2. The augmented message is split into t blocks $M_1, M_2, \ldots, M_t$ of n bits each.
3. Before the iteration starts, the chaining value H_0 is set to a fixed public value called the **IV** (initial value).
4. Iteration i combines the previous digest with the next block: $H_i = f(H_{i-1}, M_i)$ for $i = 1, \ldots, t$.
5. The last chaining value H_t is the hash of the original message, $h(M) = H_t$.

Why it is collision resistant if f is. Suppose two different messages $M \neq M'$ gave the same final digest $H_t = H'_t$. Then the last compression steps $f(H_{t-1}, M_t)$ and $f(H'_{t-1}, M'_t)$ have equal outputs. If their inputs differ, that is a collision of f. If their inputs are equal, then $H_{t-1} = H'_{t-1}$ and the same argument applies one step earlier, and so on backwards. Since $M \neq M'$, the inputs must differ at some step, so a collision of h always yields a collision of f. The length field is essential to this argument: it puts the message length in the last block, so two messages of different lengths already differ in their final inputs and cannot be lined up to share a suffix. As a result, the designer only has to make one fixed-size function collision resistant.

1.7 Compression functions: from scratch or from a block cipher ▸

▸ **Likely this year:** "Differentiate Rabin, Davies–Meyer and Miyaguchi–Preneel; why is Rabin weak?" (3 marks).

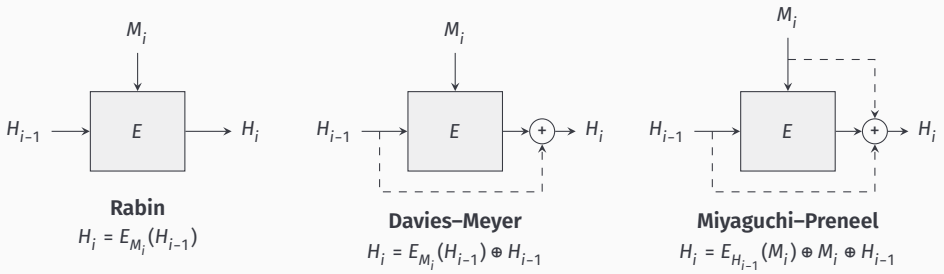
There are two ways to obtain the compression function f. In the first, f is **designed from scratch** specifically for hashing; in the second, an existing **symmetric-key block cipher** is used as f, so that the confidence gained in the cipher carries over.

Made from scratch: the MD and SHA families. The **Message Digest** family by Rivest has several versions (MD2, MD4, MD5). MD5 is a strengthened MD4 that divides the message into 512-bit blocks and creates a 128-bit digest; this digest turned out to be too short to resist collision attacks. The **Secure Hash Algorithm** family, the NIST standard also called the Secure Hash Standard (SHS), started with SHA (later SHA-1) and added SHA-224, SHA-256, SHA-384 and SHA-512 (together called SHA-2). All of them are Merkle–Damgård constructions.

Characteristic	MD5	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Maximum message size (bits)	any*	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size (bits)	512	512	512	512	1024	1024
Digest size (bits)	128	160	224	256	384	512
Number of rounds / steps	64	80	64	64	80	80
Word size (bits)	32	32	32	32	64	64

The SHA columns are the slide's table (slide 41); the MD5 column is added for comparison (*MD5 records the length modulo 2^{64}, so it accepts any length). The pattern to remember is that the 32-bit-word members use 512-bit blocks and the 64-bit-word members (SHA-384, SHA-512) use 1024-bit blocks; SHA-384 is SHA-512 with a different IV and a truncated digest.

Made from a block cipher. Let $E_K(P)$ be a block cipher with key K and plaintext P. One input of f is the message block M_i and the other is the chaining value H_{i-1}; the schemes differ in which goes into the key, which into the plaintext, and what is XORed onto the output.



In the Rabin and Davies–Meyer pictures the block M_i enters as the *key* (from above) and H_{i-1} as the plaintext; in Miyaguchi–Preneel it is the other way round: M_i is the plaintext and H_{i-1} the key (slide 45). Dashed lines are the feed-forward XORs.

Rabin scheme. Rabin plugs a block cipher directly into Merkle–Damgård: the message block is used as the key, the previous digest as the plaintext, and the ciphertext is the new digest, $H_i = E_{M_i}(H_{i-1})$. The digest size equals the cipher's block size, so with DES the digest is only 64 bits. Although the scheme is very simple, it is subject to a **meet-in-the-middle attack**. Because Eve knows every key (the message blocks are her choice), each step can be run *backwards*: $H_{i-1} = D_{M_i}(H_i)$. To forge a message with a given digest H_t, she encrypts forward from H_0 with about $2^{m/2}$ random choices of the first half of the blocks, stores the intermediate values, then decrypts backward from H_t with about $2^{m/2}$ random choices of the second half, and looks for a common intermediate value. A match gives a full message with digest H_t after only about $2 \times 2^{m/2}$ operations instead of 2^m; for DES that is about 2^{32}, which is feasible.

Davies–Meyer scheme. Davies–Meyer is the Rabin scheme plus a **feed-forward**: the previous digest is XORed onto the cipher output, $H_i = E_{M_i}(H_{i-1}) \oplus H_{i-1}$. Since H_{i-1} now appears both inside the encryption and in the XOR, knowing H_i and M_i no longer lets Eve compute H_{i-1} by decryption, so the backward half of the meet-in-the-middle attack is blocked. This is the protection the slide refers to.

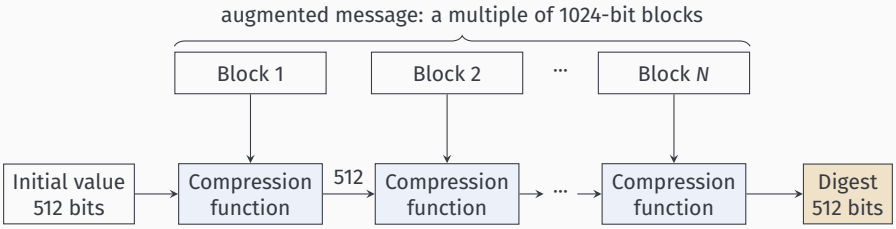
Matyas–Meyer–Oseas scheme (Forouzan, not on the slides). This is the dual of Davies–Meyer: the previous digest is the key (passed through a function g that adapts it to the key size) and the message block is the plaintext, with the block fed forward, $H_i = E_{g(H_{i-1})}(M_i) \oplus M_i$. It is useful when the key and block sizes of the cipher are equal.

Miyaguchi–Preneel scheme. To make the algorithm stronger, the plaintext, the cipher key and the ciphertext are all XORed together to create the new digest: $H_i = E_{H_{i-1}}(M_i) \oplus M_i \oplus H_{i-1}$. It is Matyas–Meyer–Oseas with the chaining value also fed forward. Whirlpool (below) is built on this scheme.

1.8 SHA-512: overview and parameters ▸

▸ **Likely this year:** "Explain SHA-512 with a block diagram: padding, word expansion, round function" (4–5 marks).

SHA-512 is the member of the SHA-2 family with the longest digest. It follows the Merkle–Damgård scheme exactly: the message is padded and given a length field, split into 1024-bit blocks, and each block is fed with the previous 512-bit digest into a compression function; the output of the last block is the 512-bit digest.



SHA-512 is **word oriented**: a word is 64 bits. Each 1024-bit message block is 16 words, and the 512-bit digest is 8 words, named A, B, C, D, E, F, G, H. The parameters are: maximum message length $2^{128} - 1$ bits, block 1024 bits, digest 512 bits, 80 rounds per block, 64-bit words, 80 round constants $K_0, \ldots, K_{79}$, and 80 expanded words $W_0, \ldots, W_{79}$ per block.

1.9 SHA-512: padding and length field ▸

▸ **Likely this year:** "Compute the number of padding bits for a message of length L; is padding needed if the length is already a multiple of 1024?" (2–3 marks).

Before processing, Merkle–Damgård requires the message to be a whole number of blocks, with its length recorded at the end. SHA-512 therefore appends two things to the original message M: **padding**, which is a single 1 bit followed by as many 0 bits as needed, and a **128-bit length field** holding $|M|$ as an unsigned integer (this is why the maximum message length is $2^{128} - 1$). The total $|M| + |P| + 128$ must be a multiple of 1024, so

$$|M| + |P| + 128 \equiv 0 \pmod{1024} \implies \boxed{|P| = (-|M| - 128) \bmod 1024}, \qquad 0 \le |P| \le 1023.$$

Since the mod gives a value from 0 to 1023, the padding is always less than one block.

WORKED EXAMPLE : padding (Forouzan Example 12.1)

Q. What is the number of padding bits if the length of the original message is 2590 bits?
A. $|P| = (-2590 - 128) \bmod 1024 = (-2718) \bmod 1024$. Since $-2718 + 3 \times 1024 = 354$, $|P|$ = **354** bits: a 1 followed by 353 zeros.
Check: $2590 + 354 + 128 = 3072 = 3 \times 1024$, so the augmented message is exactly 3 blocks.

WORKED EXAMPLE : padding, new example

Q. A message of 5000 bits is hashed with SHA-512. Find the number of padding bits, the number of blocks, and the content of the length field.
A. $|P| = (-5000 - 128) \bmod 1024 = (-5128) \bmod 1024 = -5128 + 6 \times 1024$ = **1016** bits (a 1 followed by 1015 zeros). Total = $5000 + 1016 + 128 = 6144$ = **6 blocks**. The 128-bit length field holds $5000 = 1388$ in hex, i.e. 115 zero bits followed by `1001110001000`.
Check: $6144/1024 = 6$ exactly.

Edge cases (Forouzan Examples 12.2–12.3). A message whose length is *already* a multiple of 1024 still needs padding, because the length field must be added: for $|M| = 1024$, $|P| = (-1152) \bmod 1024 = 896$ and the result is 2 blocks. The formula gives the **minimum** padding $|P| = 0$ when $|M| \equiv 896 \pmod{1024}$ (the message plus the length field already fill the block) and the **maximum** $|P| = 1023$ when $|M| \equiv 897 \pmod{1024}$. In the actual standard (FIPS 180) the single 1 bit is always appended, so the padding is at least 1 bit; for $|M| \equiv 896$ this forces a full extra block of 1024 padding bits. A short check of the real rule: the 24-bit message "abc" gets $(-24 - 128) \bmod 1024 = 872$ padding bits and fits in one block.

Trap: padding arithmetic

Take the mod of a negative number correctly: add multiples of 1024 until the value lies in 0–1023. Do not forget the 128-bit length field (SHA-1 and SHA-256 use a 64-bit field and 512-bit blocks, giving $|P| = (-|M| - 64) \bmod 512$).

1.10 SHA-512: word expansion and the compression function ▸

Word expansion. Each round uses one 64-bit word, but a block has only 16 words, so the block is expanded to 80 words $W_0, \ldots, W_{79}$. The first 16 are the block itself; each later word mixes four earlier ones:

$$W_i = W_{i-16} \oplus \text{RotShift}_{1\text{-}8\text{-}7}(W_{i-15}) \oplus W_{i-7} \oplus \text{RotShift}_{19\text{-}61\text{-}6}(W_{i-2}), \qquad 16 \le i \le 79,$$

$$\text{RotShift}_{l\text{-}m\text{-}n}(x) = \text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x),$$

where $\text{RotR}_i(x)$ rotates x right by i bits (bits leaving on the right re-enter on the left) and $\text{ShL}_i(x)$ shifts x left by i bits, filling with 0s (bits leaving are lost). This is the slides' (Forouzan's) form; see the correction box for the standard's exact operations. Since every later word depends on several earlier ones, a change in one message bit spreads into many of the 80 words.

Initial values and constants. The eight words of H_0 are the first 64 bits of the fractional parts of the square roots of the first eight primes (2 to 19); for example A_0 = `6A09E667F3BCC908` from $\sqrt{2}$. The 80 round constants $K_0, \ldots, K_{79}$ are the first 64 bits of the fractional parts of the cube roots of the first 80 primes (2 to 409); for example K_0 = `428A2F98D728AE22` from $\sqrt[3]{2}$. Taking them from irrational numbers shows that the designers hid no trapdoor in the constants ("nothing up my sleeve" numbers).

Compression function. Processing one block takes 80 rounds. At the start, the eight digest words from the previous block (or the IV) are saved in eight temporary variables. Round i mixes the current $A, \ldots, H$ with the word W_i and the constant K_i and produces a new set of eight words. After round 79, the saved values are added to the outputs word by word modulo 2^{64}; this **final adding** produces the digest that is passed to the next block (or

is the final digest). The final adding is a feed-forward, exactly like Davies–Meyer, and it makes the compression function impossible to run backwards.

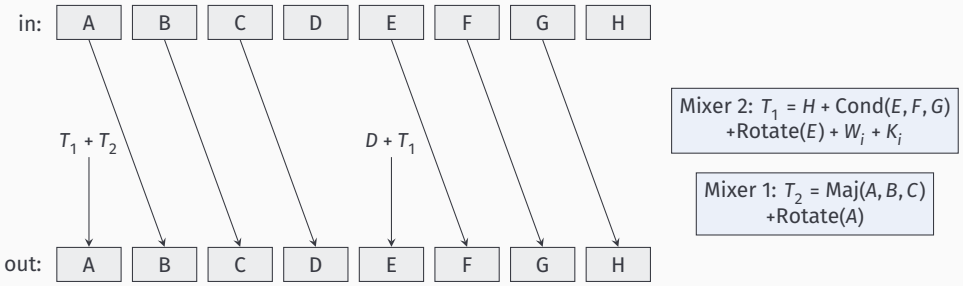
Structure of each round. Six of the eight words are simply moved one place right, and only two new words are computed, using two mixers (every + here is addition modulo 2^{64}):

$$T_1 = H + \text{Conditional}(E,F,G) + \text{Rotate}(E) + W_i + K_i, \qquad T_2 = \text{Majority}(A,B,C) + \text{Rotate}(A),$$

$$H \leftarrow G,\ G \leftarrow F,\ F \leftarrow E,\ E \leftarrow D + T_1,\ D \leftarrow C,\ C \leftarrow B,\ B \leftarrow A,\ A \leftarrow T_1 + T_2.$$

The three functions are bitwise (except Rotate, which moves bits within the word):

- Majority$(x,y,z) = (x \wedge y) \oplus (y \wedge z) \oplus (z \wedge x)$: each output bit is the majority value of the three input bits.
- Conditional$(x,y,z) = (x \wedge y) \oplus (\overline{x} \wedge z)$: each bit of x acts as a selector, taking the bit of y where $x = 1$ and the bit of z where $x = 0$ ("if x then y else z").
- Rotate$(x) = \text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$.



In Forouzan's notation the new A is called X and the new E is Y. Because A and E change every round and the other six words are shifted, every input word affects all eight words within a few rounds.

★ MUST KNOW : SHA-512 in numbers

Block 1024 bits = 16 words of 64 bits; digest 512 bits = 8 words A–H; 80 rounds; 80 words W_0–W_{79}; 80 constants K_i; length field 128 bits; $|P| = (-|M| - 128) \bmod 1024$; max message $2^{128} - 1$ bits; final adding modulo 2^{64}.

1.11 Exercises on the SHA-512 operations ▸

▸ **Likely this year:** One of the slide exercises (RotR, ShL, Conditional, Majority, Rotate) on given hex words.

The slide exercises use a **128-bit** x (real SHA-512 words are 64 bits) and work in hex: a rotation or shift by 12 bits moves the string by 3 hex digits. All results below were recomputed.

$$x = 1234\ \ 5678\ \ ABCD\ \ 2345\ \ 3456\ \ 5678\ \ ABCD\ \ 2468$$

WORKED EXAMPLE : RotR, ShL, Conditional, Majority, Rotate (slides 52–56)

$\text{RotR}_{12}(x)$. Rotate right by 12 bits = 3 hex digits: the last three digits 468 move to the front.
4681 2345 678A BCD2 3453 4565 678A BCD2
$\text{ShL}_{12}(x)$. Shift left by 3 hex digits: the first three digits 123 are lost and three 0 digits enter on the right.
4567 8ABC D234 5345 6567 8ABC D246 8000
Conditional(x,y,z) with y = 2234 5678 ..., z = 3234 5678 ... (the three words differ only in the first digit). For the first digit, x = 0001 selects y where $x = 1$ and z where $x = 0$: $(0001 \wedge 0010) \oplus (1110 \wedge 0011) = 0000 \oplus 0010 = 0010 = 2$. For every other digit the three inputs are equal, and Conditional of three equal digits is that digit.
Result: 2234 5678 ABCD 2345 3456 5678 ABCD 2468
Majority(x,y,z), same x,y,z. First digit: $(0001 \wedge 0010) \oplus (0010 \wedge 0011) \oplus (0011 \wedge 0001) = 0000 \oplus 0010 \oplus 0001 = 0011 = 3$. The other digits are unchanged.
Result: 3234 5678 ABCD 2345 3456 5678 ABCD 2468
Rotate$(x) = \text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$. First the three rotations, then the XOR:

$\text{RotR}_{28}(x)$	BCD2	4681	2345	678A	BCD2	3453	4565	678A
$\text{RotR}_{34}(x)$	2AF3	491A	048D	159E	2AF3	48D1	4D15	959E
$\text{RotR}_{39}(x)$	F157	9A48	D024	68AC	F157	9A46	8A68	ACAC
Rotate(x)	6776	95D3	F7EC	1AB8	6776	E6C4	8218	5EB8

RotR_{28} is a rotation by 7 hex digits, so it can be done digit-wise; RotR_{34} and RotR_{39} are not multiples of 4 and must be done in binary.

WORKED EXAMPLE : new exercise on 16-bit and 64-bit words

Q. (a) For the 16-bit words x = F0F0, y = AAAA, z = 5555, find Conditional(x, y, z) and Majority(x, y, z). (b) For the 64-bit word w = 0123 4567 89AB CDEF, find $\text{RotR}_8(w)$ and $\text{ShL}_8(w)$.

A. (a) Conditional takes y where x is 1 and z where x is 0. $x \wedge y$ = A0A0, $\overline{x} \wedge z$ = 0F0F $\wedge$ 5555 = 0505; Conditional = A0A0 $\oplus$ 0505 = **A5A5**.
Majority: $x \wedge y$ = A0A0, $y \wedge z$ = 0000 (since $z = \overline{y}$), $z \wedge x$ = 5050; XOR = **F0F0**. This equals x, as it must: y and z disagree in every bit, so x always casts the deciding vote.
(b) 8 bits = 2 hex digits. $\text{RotR}_8(w)$ = EF01 2345 6789 ABCD (EF wraps to the front); $\text{ShL}_8(w)$ = 2345 6789 ABCD EF00 (01 is lost, 00 enters).

1.12 Whirlpool

Whirlpool, designed by Rijmen and Barreto and selected by the NESSIE project (New European Schemes for Signatures, Integrity, and Encryption), is an iterated hash function based on the **Miyaguchi–Preneel** scheme, in which the compression function is a block cipher: $H_i = E_{H_{i-1}}(M_i) \oplus M_i \oplus H_{i-1}$. The cipher, called W, is a **modified AES** tailored for this purpose.

Parameters. Block size 512 bits, digest 512 bits, cipher key 512 bits (the previous digest), initial value H_0 all 0s, and maximum message length $2^{256} - 1$ bits. The message is padded (a 1 followed by 0s) to an *odd* multiple of 256 bits and then a **256-bit length field** is appended; an odd multiple of 256 plus 256 is an even multiple, i.e. a multiple of 512. For example, $|M| = 1000$ needs $|P| = (-1000 - 256) \bmod 512 = 280$ padding bits, since $1000 + 280 = 1280 = 5 \times 256$ (odd) and $1280 + 256 = 1536 = 3 \times 512$.

How the Whirlpool cipher differs from AES. It uses the same four kinds of transformation, but on a larger state and with all rounds alike:

- The block and key are 512 bits, held as an 8×8 matrix of bytes (AES: 128 bits as 4×4).
- There are 10 rounds, each with SubBytes, **ShiftColumns** (column j is rotated down by j positions, instead of AES's ShiftRows), **MixRows** (each row is multiplied by a fixed 8×8 circulant matrix, instead of AES's MixColumns) and AddRoundKey.
- Arithmetic is in $\text{GF}(2^8)$ with modulus $x^8 + x^4 + x^3 + x^2 + 1$ (AES uses $x^8 + x^4 + x^3 + x + 1$), and the S-box is built from smaller 4-bit boxes.
- The round keys are generated by running the same round function on the key, with round constants taken from the S-box in place of a key, instead of AES's separate key-expansion routine.

Corrections to the notes / slides

- Slide 27: "0.69×2^{34} seconds or more than 500 years". In fact 0.69×2^{34} s $\approx$ 376 years (2^{34} s alone is $\approx$ 544 years). The conclusion (secure against preimage attacks) stands.
- Slides 54–55 swap the binary for the first digit: Conditional(0001, 0010, 0011) = 0010 = 2 and Majority(0001, 0010, 0011) = 0011 = 3; the slides print 0011 for Conditional and 0010 for Majority, though the hex results 2 and 3 are right. Both slides also label the third word x instead of z.
- The exercise x (slides 52–56) is 128 bits, though real SHA-512 words are 64 bits; the rotations on the slides are done on 128 bits, and the results above follow the slides.
- Slide 11 prints Problem 4 as $P \approx 1 - e^{-k^2/2N}$; that formula would give $k = 1.18\sqrt{N}$. The correct one is $P \approx 1 - e^{-k^2/N}$ (as on slide 35), which gives $k = 0.83\sqrt{N} = 16$.
- Slide 40 lists SHA under the "block cipher" approach. Forouzan classifies MD and SHA as compression functions *made from scratch* and Rabin, Davies–Meyer, Matyas–Meyer–Oseas and Miyaguchi–Preneel as block-cipher based; write it that way in the exam.
- Slides 48 and 51 (from Forouzan) simplify the standard: in FIPS 180 the word expansion uses addition modulo 2^{64} and a *right* shift (SHR), not XOR and ShL, and Rotate(E) uses $\text{RotR}_{14} \oplus \text{RotR}_{18} \oplus \text{RotR}_{41}$, not 28-34-39. The slide's "ShL...padding the left by 0s" is self-contradictory (a left shift fills on the right). For the exam, use the slide formulas.

Check your understanding

1. **Differentiate preimage, second preimage and collision resistance.** Preimage resistance: given only a digest y, Eve cannot find any message hashing to y. Second preimage resistance: given a message M and its digest, she cannot find a different message with the same digest. Collision resistance: she cannot find any two different messages with the same digest, choosing both herself. For an n-bit ideal hash the first two cost about 2^n and the third about $2^{n/2}$.
2. **Why does collision resistance imply second preimage resistance?** If Eve could find a second preimage for a given M, she could choose any M, find M', and output (M, M') as a collision. So breaking second preimage resistance breaks collision resistance, and a collision-resistant function must be second-preimage resistant.
3. **Why can a checksum not serve as a cryptographic hash?** A checksum is linear and easy to invert, so given

a target value Eve can adjust any message to reach it; it is therefore not preimage resistant. It only detects accidental errors.

4. **Why can the random oracle not use the formula $h(M) = M \bmod n$?** Because then $h(M_1+M_2) = [h(M_1)+h(M_2)] \bmod n$ can be predicted from earlier digests, violating the rule that each new digest is chosen independently at random.
5. **Derive the probability that among k random n-bit digests two are equal.** All k differ with probability $\prod_{i=1}^{k-1}(1 - i/N) \approx e^{-k(k-1)/2N}$, $N = 2^n$, so $P \approx 1 - e^{-k(k-1)/2N}$; setting $P = 1/2$ gives $k \approx 1.18\sqrt{N} = 1.18 \times 2^{n/2}$.
6. **How many digests must Eve create to find a collision in a 64-bit hash with $P > 0.5$?** $k = 1.18 \times 2^{32} \approx 5.07 \times 10^9$; at 2^{20} per second this takes $1.18 \times 2^{12} \approx 4833$ s, about 1.3 hours, so 64 bits is insecure against collisions (a preimage would take about 376 years at 2^{30} per second).
7. **What digest length gives 2^{128} security against collisions?** The collision attack costs about $2^{n/2}$, so $n/2 = 128$ and $n = 256$ bits (SHA-256).
8. **Compute the SHA-512 padding for $|M| = 3000$ bits.** $|P| = (-3000 - 128) \bmod 1024 = -3128 + 4 \times 1024 = 968$ bits; total $3000 + 968 + 128 = 4096 = 4$ blocks.
9. **Why is the Rabin scheme weak, and how does Davies–Meyer fix it?** In Rabin, $H_i = E_{M_i}(H_{i-1})$ and Eve knows every key, so she can decrypt backwards from a target digest and meet a forward computation in the middle with about $2^{m/2}$ work each way. Davies–Meyer XORs H_{i-1} onto the output, so H_{i-1} can no longer be recovered from H_i and M_i, which blocks the backward half.
10. **Why is the length field appended in Merkle–Damgård?** It makes the last block depend on the message length, which lets a collision of the whole hash be traced back to a collision of the compression function; without it, messages of different lengths could share a suffix of chaining values.